

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\agent-a636c498146ddc3fb.jsonl`
- SHA-256: `733e40d6fbc09cd7d0d97b49b4c7f2bd9f15b3df65f3226dba6052b0f31140b3`
- Source modified: `2026-06-21T14:12:22+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T14:09:32.438Z)

I'm writing a localization project proposal for a React (react-i18next + i18next-icu) web app that today supports English (en), Kannada (kn), Hindi (hi). I need to ADD: Thai, Indonesian, Danish, Spanish, Chinese, Japanese, Korean ? with the same rigor (CLDR plurals, self-hosted OFL Noto fonts, key-parity, native-speaker review).

For EACH of those 7 languages, research and report (with authoritative sources ? Unicode CLDR plural-rules charts, Google Noto / SIL OFL, @fontsource registry, BCP-47):

1. The BCP-47 locale code(s) to use, including any SCRIPT VARIANTS that need a decision (especially Chinese: zh-Hans Simplified vs zh-Hant Traditional ? what's the standard choice + codes; and whether ja/ko/th need any subtag).
2. The CLDR CARDINAL plural categories (e.g. {one, other} vs {other} vs {one, many, other}). Be precise ? note which are single-form (other only) and any with more than 2 forms (I believe Spanish now has "many"). This determines ICU MessageFormat plural authoring.
3. Text DIRECTION (LTR/RTL) ? confirm all 7 are LTR (no RTL layout work) or flag any.
4. The FONT needed for correct rendering: which need a dedicated Noto font (name, e.g. "Noto Sans Thai", "Noto Sans SC/JP/KR") vs which are covered by a standard Latin font (Spanish, Danish, Indonesian). For each dedicated font: is it OFL-1.1 (commercially bundleable)? Is it on @fontsource / @fontsource-variable? Approx self-hosted web-font SIZE and the CJK subsetting concern (Noto CJK fonts are very large ? give rough numbers and the implications for an OFFLINE/air-gapped self-hosted bundle).
5. Any rendering/layout gotchas (CJK line-breaking, Thai no-spaces word-wrap, longer Spanish/German-like string expansion, etc.).

Then give a RECOMMENDED PHASING for adding these 7 (e.g. Latin-script-first since no new font; then Thai; then CJK because of font size + script-variant decisions), and a short note on what stays catalog-only (code-free) vs what needs new code/assets (fonts, lazy-loading).

Return a clear, well-organized markdown report with a per-language table + the phasing recommendation + sources. Be accurate and cite sources; this feeds an owner-approved project doc.

2. assistant (2026-06-21T14:09:36.324Z)

I'll research this thoroughly using the deep-research approach. Since this feeds an owner-approved project doc and requires authoritative sources (CLDR, Unicode, SIL OFL, @fontsource, BCP-47), let me first check the existing i18n setup in the repo to align with current conventions, then run the research.

3. user (2026-06-21T14:09:37.836Z)

.claude\worktrees\bhi-execpolicy\docs\I18N_PLAN.md
.claude\worktrees\agent-a771ff1d85e18f2ed\docs\I18N_PLAN.md
.claude\worktrees\tier1-windowing\docs\I18N_PLAN.md
.claude\worktrees\w2-safeguard\docs\I18N_PLAN.md

```
.claude\worktrees\hardchop\docs\I18N_PLAN.md
.claude\worktrees\boxhill17\docs\I18N_PLAN.md
.claude\worktrees\pr-stitch\docs\I18N_PLAN.md
.claude\worktrees\gap-doc\docs\I18N_PLAN.md
.claude\worktrees\resweep\docs\I18N_PLAN.md
.claude\worktrees\recdoc\docs\I18N_PLAN.md
docs\I18N_PLAN.md
```

4. user (2026-06-21T14:09:37.855Z)

No files found

5. assistant (2026-06-21T14:09:40.232Z)

Let me read the existing I18N_PLAN.md to align with the project's conventions, and pull the repo fresh per convention.

6. user (2026-06-21T14:09:40.896Z)

```
1      # Internationalization (i18n) ? Kannada-first, by design
2
3      **Status:** Design (P0 deliverable; phases P1+ are owner-gated, not started).
4      **Owner decision required** before any code lands ? see [Open decisions](#open-
decisions-for-the-owner).
5
6      > **? Implementation (2026-06-21):** this design is now being executed in the
**`khelsutra`** repo against
7      > today's surfaces ? tracked in **`khelsutra/docs/FRIENDLY_AND_MULTILINGUAL.md`** ($7
P-tracker is the source of
8      > truth). P1's "static-UI retrofit" is **retargeted**: `backend/api/static` is gone, so
**P1 ? the vanilla
9      > `site/` (`t()` shim) and P3 ? the `web/` React SPA (react-i18next)**, both consuming
shared
10     > `locales/{en,kn,hi}` catalogs co-located in `khelsutra`. The framework-agnostic
foundation below is
11     > unchanged. Backend localization (P2) stays here in the engine repo, owner-gated.
12
13     ---
14
15     ## 1. Context ? why this, why now
16
17     The PMF beachhead is **Bangalore badminton academies**
([`PMF_MARKET_RESEARCH.md`](archives/field-pmf/PMF_MARKET_RESEARCH.md)).
18     Many academy coaches, parents, and players are far more comfortable in **Kannada** than
English.
19     A product that only speaks English adds friction exactly at the point of first contact.
So the
20     product must speak **Kannada first**, **Hindi next**, and extend to other Indian
languages
21     (Tamil, Telugu, Marathi, ?) **without a refactor**.
22
23     The owner's directive is precise: make the product **internationalizable _by design_**.
Adding a
24     language must be **"add a reviewed catalog" ? not "go re-touch the code."** The
translations
25     themselves can wait; **the foundation that makes them cheap to add is the actual
deliverable.**
26
27     Today there is **zero i18n** in the repo (no i18n/locale/gettext/babel anywhere). Every
28     user-facing string is hardcoded English across four surfaces:
29
30     | Surface | Where | Examples (today) |
31     |---|---|---|
32     | **UI chrome** | `backend/api/static/{index.html,app.js,style.css}` (~40?50 strings) |
```

```

`"Validate & Index Video"`, `alert("Validation Failed! ?")`, `` ` ${count} Rally Segment${count
> 1 ? 's' : ''} Selected` `` |
33 | **Backend coach-facing text** | `backend/main.py` `HTTPException(detail=?)`;
`backend/pipeline/validators/*` `message`; `backend/reporting/spec.yaml` `customer_verdict` +
per-rule `customer_message` | `Video file not found at '?'`; `Video quality rejected: Focus
check failed?`; `"? Your highlights are ready."` |
34 | **Self-navigating NL queries** (owner ask, future) | not built yet | **show me all
rallies > 15 shots where I won** ? the **query and its answer** must work in the user's language
|
35 | **Coach-mode recommendations** (owner ask, future) | not built yet | localized
recommendation text, not English |
36
37 A React/Vite SPA is planned to replace the static UI ([`UI_ALPHA_EXECUTION_PLAN.md`],
ADR-009)
38 in a **separate private repo `avidullu/khelsutra`**. The design below is therefore
deliberately
39 **framework-agnostic**: the catalog format and locale-negotiation contract are shared,
so the
40 static UI today and the SPA tomorrow consume the *same* translation source.
41
42 **Scope confirmed with owner:** UI + coach-facing backend text + (designed-for, built-
later)
43 NL/coach features. **Translations are Gemini-drafted, human-reviewed** ? this is
*inference*
44 use of a paid LLM, which the [licensing guardrail](#guardrail-alignment) explicitly
allows
45 (it is *not* a training-data source). The owner reviews the first draft of every string.
46
47 ---
48
49 ## 2. Core architecture
50
51 ### 2.1 The "by design" invariant ? no hardcoded user-facing strings
52
53 Every coach-facing string is a **stable key** resolved at render time against the active
locale.
54 Code references `t("compile.button")`, never the English prose. This is the single rule
that
55 makes "add a language = add a catalog" true, and it is **enforced by CI** ($2.6) so it
cannot
56 silently regress as the codebase grows.
57
58 ```
59 "Validate & Index Video" ? t("video.action.validate_index")
60 alert("Validation Failed! ?review the checklist") ? t("toast.validation_failed")
61 ` ${count} Rally Segment${count>1?'s':''} Selected` ? t("compile.selected", {count}) #
ICU plural
62 HTTPException(detail="Video file not found at ?")? detail = tr(lang,
"error.video_not_found", {path})
63 ```
64
65 ### 2.2 One canonical English source catalog, per-locale siblings
66
67 ```
68 locales/
69   en/  common.json  # source of truth ? devs author keys here
70   kn/  common.json  # Kannada (Gemini-draft ? human-review)
71   hi/  common.json  # Hindi (next; first exercise of the extensibility checklist)
72 ```
73
74 - **Format: i18next-flavored JSON** ? works for the vanilla shim today *and*
react-i18next in
75 the SPA, with no conversion.
76 - **ICU MessageFormat** for interpolation and plurals: `{count, plural, one {# rally}
other {# rallies}}`.

```

```

77     This matters immediately ? the current UI already hand-rolls English plurals
78     (`Rally Segment${count > 1 ? 's' : ''}`), and Kannada/Hindi plural rules differ from
English.
79     ICU encodes each language's rules in the catalog, not the code.
80     - `en` is the source of truth; every other locale is keyed identically (enforced by
the
81         key-parity check in §2.6).
82
83     ### 2.3 Locale-negotiation contract (shared frontend ? backend)
84
85     One resolution order, defined in one place and honored by both UI and backend:
86
87     ```
88     persisted user preference ? UI switcher / ?lang=<code> ? Accept-Language header ?
default "en"
89     ```
90
91     - Supported locales are enumerated in a single module (one list to grow). Anything
outside
92         it falls back to `en`.
93     - The backend localizes its own responses (`HttpException.detail`, report `customer_`)
from the
94         same `lang` query param / `Accept-Language` header, so a Kannada UI gets Kannada
errors.
95     - Always-fallback to `en` for any missing key ? a missing translation must degrade
to
96         English, never to a blank or a raw key in front of a user.
97
98     ### 2.4 Backend text ? keys + params, not prose
99
100    Validator `message`s and API errors today bake in English prose and interpolated
values
101    (`f"?sharpness score of {avg} is below ? threshold of {threshold}.")`. The design
separates
102    the two:
103
104    - The validator/endpoint emits a stable code + a params dict
(`("blur.below_threshold",
105        {"score": avg, "threshold": threshold})`).
106    - The prose lives in the catalog, looked up at the API boundary in the negotiated
locale.
107    - `ValidationResult` keeps an English `message` for logs/back-compat, but gains an
optional
108        `message_key` + `message_params`; the report/serializer prefers the key when present.
This
109        keeps internal/technical messages English (good for debugging) while customer-facing
text
110        localizes.
111
112    `spec.yaml` `customer_verdict` and per-rule `customer_message` become keyed (or per-
locale
113    overlays keyed by rule `code`); the technical `message` blocks stay English (they cite
114    `README#Qn` and are for the operator, not the end user).
115
116    ### 2.5 Indic font strategy ? (open decision ? see §4)
117
118    The bundled UI/watermark fonts (Outfit, Roboto Slab) do not cover Kannada or
Devanagari.
119    Noto Sans Kannada / Noto Sans Devanagari are OFL-licensed, which is the same tension
we hit
120    with the watermark font: OFL is not in the strict MIT/Apache/BSD code/data/weights
guardrail.
121    Fonts are media assets, not code or model weights, and OFL is explicitly designed for
122    unrestricted redistribution ? so the cleanest path is to carve out an explicit OFL-
for-fonts

```

```

123     exception**, the same way we'd treat any bundled media. Options put to the owner in §4.
124
125     Note the **video watermark already takes a `font_path`** (`WatermarkConfig.font_path`,
126     `backend/pipeline/stitcher/compiler.py`), so stamping a Kannada/Hindi watermark is a
127     *configuration* change (point it at a Noto font) once the font decision is made ? no new
code.
128
129     ### 2.6 CI enforcement (the guardrail that keeps it "by design")
130
131     Two checks, both pure-Python and runnable in this repo (no GPU/torch needed):
132
133     1. **Key-parity test** ? every locale catalog has *exactly* the `en` key set; fail on
missing
134         keys, extra keys, or untranslated placeholder markers (e.g. a ` "__MISSING__" `
sentinel left
135         by the draft script). This is also the **extensibility proof**: drop a stub `ta.json`
with
136         one key and CI immediately lists every gap.
137     2. **No-bare-strings lint** ? a check that flags user-facing string literals not routed
through
138         `t()` / the backend `tr()` (scoped to the UI handlers and the API/validator/report
text
139         paths). Starts as a curated allowlist so it can land without boiling the ocean.
140
141     ### 2.7 Translation pipeline
142
143     ```
144     dev authors en keys ? scripts/i18n_draft.py (Gemini, INFERENCE only) ? kn/hi draft
145                          ? native-speaker review (owner reviews first draft)
146                          ? CI key-parity + placeholder check ? merge
147     ```
148
149     `scripts/i18n_draft.py` reads `locales/en/*.json`, asks Gemini for each missing key in
the
150     target locale **with the key name and any ICU placeholders preserved**, and writes a
draft
151     catalog with every machine-drafted value flagged for review. It never touches `en`,
never
152     invents keys, and is a *drafting aid* ? a human reviews before merge. (Reuses the
existing
153     Gemini provider credentials/config; this is the same "paid LLM = inference, never
training"
154     posture as the rest of the system.)
155
156     ---
157
158     ## 3. Phased execution (owner-gated; ONE PR per slice, off `master`)
159
160     | Phase | Scope | Repo | Gate |
161     |---|---|---|---|
162     | **P0** | **This design doc** (`docs/I18N_PLAN.md`) | this | _the immediate
deliverable_ |
163     | **P1** | Foundation + **static-UI retrofit**: `locales/` + `en` source catalog
(extract the ~40?50 UI strings ? keys), a tiny vanilla **`t(key, vars)`** shim + `data-i18n`
wiring in `index.html/app.js`, **language switcher (en/kn)** + `localStorage` persistence +
`?lang=`, the font decision applied, and the **`kn`** catalog (Gemini-draft ? owner review).
Reuse the existing static mount / `serve_index` in `backend/main.py`. | this | owner approval |
164     | **P2** | **Backend + report localization**: `backend/i18n/` with `tr(lang, key,
params)` (same catalog format), refactor validator `message`s to keys+params, localize
`HTTPException.detail` + `spec.yaml` `customer_verdict`/`customer_message`, wire `Accept-
Language`/`?lang=`. | this | owner approval |
165     | **P3** | **SPA adoption**: **react-i18next** consuming the *same* `locales/` catalogs;
documented FE?BE contract. | `avidullu/khelsutra` | needs repo access; or owner/Opus executes
there |
166     | **P4** | **Conversational + coach-mode** (future): locale-aware NL query intake ?

```

```

**templated localized answers**; coach-mode recommendation **templates**. Ties to the owned-
model / PLATFORM $5A + rich-metadata work. | this + model | gated on those features existing |
167
168     **Extensibility checklist** (this *is* the by-design contract ? runs the same every
time):
169     > add `locales/<code>/common.json` ? `scripts/i18n_draft.py --locale <code>` ? human
review ?
170     > add `<code>` to the supported-locales list ? confirm font coverage ? CI key-parity
green.
171
172     **Hindi (`hi`) is the first real exercise of this checklist** ? if adding `hi` requires
touching
173     anything other than the steps above, the design failed and we fix the design, not paper
over it.
174
175     ---
176
177     ## 4. Open decisions (for the owner)
178
179     1. **Target-UI scope** ? confirm the *shared framework-agnostic foundation* (prove on
the static
180     UI now, SPA adopts the same catalogs later). Alternative: scope only to the SPA and
skip the
181     static-UI retrofit. _Recommendation: shared foundation ? it's the cheap, low-risk way
to
182     validate the catalog/negotiation contract before the SPA exists._
183     2. **Indic font licensing** ? (a) **allow OFL for bundled fonts** and self-host Noto
Sans
184     Kannada/Devanagari _(recommended; treats fonts as media, not code/weights); (b) rely
on
185     **system Indic fonts** (no bundle, inconsistent rendering across machines); (c)
**Google Fonts
186     CSS** (runtime external dependency, privacy/offline concerns for a local-first
product). This
187     also unblocks the **Kannada/Hindi video watermark** (needs a Noto `font_path`).
188     3. **v1 locale set & fallback** ? confirm `en` + `kn` for v1, `hi` next; always-fallback
to `en`
189     for missing keys.
190     4. **Backend-localization depth for v1** ? recommendation: localize the **customer
verdict /
191     report `customer_***` and **validation rejection reasons** first (highest user
impact);
192     defer technical/operator messages (they stay English by design).
193
194     ---
195
196     ## 5. Key files / seams
197
198     - **Localize:** `backend/api/static/{index.html,app.js,style.css}`; `backend/main.py`
199     (`HTTPException.detail` + the `serve_index`/static mount);
`backend/pipeline/validators/`
200     (`message` ? key+params; see `validators/base.py::ValidationResult`);
201     `backend/reporting/spec.yaml` (`customer_verdict` + per-rule `customer_message`).
202     - **New:** `locales/{en,kn,hi}/`, `backend/i18n/` (`tr()` + supported-locales +
negotiation),
203     `scripts/i18n_draft.py`, a CI key-parity test under `tests/` + a no-bare-strings check
wired
204     into `.github/workflows/ci.yml`.
205     - **Fonts:** reuse the bundled-asset pattern in `backend/assets/fonts/` (Apache-2.0
Roboto Slab
206     ships there today); add Noto Indic fonts there *iff* decision $4.2(a).
207     - **SPA:** `avidullu/khelsutra` (react-i18next) ? separate repo.
208
209     ---
210

```

```

211     ## 6. Verification
212
213     - **CI key-parity test** (Python, this repo): every locale has every `en` key; fail on
214       missing/extra/placeholder. Drop a stub `ta.json` ? CI flags every gap (the
215       extensibility proof).
216     - **Static UI:** load `?lang=kn` ? rendered strings resolve from the `kn` catalog;
217       switcher
218       persists across reload; any missing key falls back to English (no raw keys shown).
219     - **Backend:** request with `Accept-Language: kn` (or `?lang=kn`) ? localized error
220       `detail` +
221       report `customer_`; unit tests for fallback, ICU interpolation, and plural rules.
222     - **Fonts:** Kannada + Devanagari render correctly ? **manual eyeball on the owner's
223       machine**
224       (the cloud container can't render fonts).
225     - **Docs-only P0 PR:** internal links resolve; indexed in [`docs/README.md`](README.md);
226       cross-linked from [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) (the NL/coach
227       surfaces)
228       and [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (the Bangalore beachhead).
229     ---
230
231     ## Guardrail alignment
232
233     - **Paid LLM = inference only.** Gemini drafts translations (inference); it is **never**
234       a
235       training-data source. ? consistent with [`CLAUDE.md`](../CLAUDE.md) and the ratified
236       guardrail.
237     - **Licensing.** Code stays MIT/Apache-2.0/BSD. The *only* tension is **fonts** (OFL) ?
238       handled
239       as an explicit, owner-approved media-asset exception in §4.2, not a silent dependency.
240     - **No minors / biometrics scope creep.** i18n touches presentation only; it introduces
241       no new
242       personal-data collection.
243
244     [`UI_ALPHA_EXECUTION_PLAN.md`]: ./UI_ALPHA_EXECUTION_PLAN.md
245
246

```

7. assistant (2026-06-21T14:09:47.210Z)

I now understand the project's existing conventions thoroughly. The current setup uses i18next-flavored JSON + ICU MessageFormat, self-hosted OFL Noto fonts, CI key-parity, and Gemini-draft ? native-review. My research must extend this pattern to 7 new languages.

Let me run the research with parallel web searches against authoritative sources.

8. user (2026-06-21T14:09:54.858Z)

Web search results for query: "Unicode CLDR cardinal plural rules Thai Indonesian Spanish Chinese Japanese Korean Danish categories"

Links: [{"title": "Language Plural Rules", "url": "https://www.unicode.org/cldr/charts/48/supplemental/language_plural_rules.html"}, {"title": "Unicode CLDR Plural Rules 2026: Categories, Examples & ICU", "url": "https://intlpull.com/blog/cldr-plural-rules-complete-guide-2026"}, {"title": "Plural Rules", "url": "https://cldr.unicode.org/index/cldr-spec/plural-rules"}, {"title": "Proposed Unicode Technical Standard #35: Unicode Locale Data Markup Language (LDML) Part 3: Numbers", "url": "https://www.unicode.org/reports/tr35/dev/tr35-numbers.html"}, {"title": "Unicode CLDR - Plural Rules", "url": "https://sites.google.com/unicode.org/cldr/index/cldr-spec/plural-rules"}, {"title": "OLD! CLDR - Unicode Common Locale Data Repository - CLDR 28 Release Note", "url": "https://sites.google.com/site/cldr/cldr-releasesdownloads/cldr-28-release-note"}, {"title": "Unicode CLDR - CLDR 1.6 Release Note", "url": "https://cldr.unicode.org/index/downloads/cldr-1-6-release-note"}, {"title": "Unicode Locale Data Markup Language (LDML) Part 3: Numbers", "url": "https://cldr-smoke.unicode.org/spec/main/ldml/tr35-numbers.html"}, {"title": "CLDR Charts", "url": "https://cldr.unicode.org/index/charts"}, {"title": "GitHub - xyzsd/cldr-plural-rules: CLDR plural rules, simplified · GitHub", "url": "https://github.com/xyzsd/cldr-plural-"}]

```
rules"]}]
```

Based on the search results, here's what I found about Unicode CLDR cardinal plural rules for the languages you mentioned:

Overview of CLDR Plural Rules

CLDR defines six plural categories?zero, one, two, few, many, and other?which are selectively used by different languages based on their grammatical rules.

Language-Specific Cardinal Plural Rules

****Asian Languages (Thai, Indonesian, Chinese, Japanese, Korean):****

Japanese, Chinese, Korean, Vietnamese, and several other Asian languages don't grammatically distinguish between singular and plural, so they use only the "other" category for all counts. This means Thai and Indonesian may have similar characteristics to other Asian languages in not requiring complex plural distinctions.

****European Languages (Spanish, Danish):****

While the search results don't provide specific details about Spanish and Danish cardinal plural categories, they do show that improvements to RBNF (number spellout) rules were made for many languages including Danish, Spanish, and Thai.

How Plural Categories Work

The categories are determined by changes required in a phrase or sentence if a numeric placeholder changes value, represented by {0}, and will be replaced by a series of digits. The fundamental rule for determining plural categories is the existence of minimal pairs: whenever two different numbers may require different versions of the same message, then the numbers have different plural categories.

For detailed language-specific rules for each language you mentioned, the official CLDR Language Plural Rules chart at unicode.org would provide the complete categorization.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

9. user (2026-06-21T14:09:55.509Z)

Web search results for query: "CLDR Spanish cardinal plural "many" category compact long form one many other"

Links: [{"title":"Plural Rules","url":"https://cldr.unicode.org/index/cldr-spec/plural-rules"}, {"title":"Language Plural Rules","url":"https://www.unicode.org/cldr/charts/48/supplemental/language_plural_rules.html"}, {"title":"Plurals & Units","url":"https://cldr.unicode.org/translation/getting-started/plurals"}, {"title":"Unicode CLDR Plural Rules 2026: Categories, Examples & ICU","url":"https://intlpull.com/blog/cldr-plural-rules-complete-guide-2026"}, {"title":"Moved","url":"https://unicode-org.github.io/cldr-staging/charts/37/supplemental/language_plural_rules.html"}, {"title":"GitHub - xyzsd/cldr-plural-rules: CLDR plural rules, simplified · GitHub","url":"https://github.com/xyzsd/cldr-plural-rules"}, {"title":"OLD! CLDR - Unicode Common Locale Data Repository - Plurals & Units","url":"https://sites.google.com/site/cldr/information-hub-for-linguists/getting-started/plurals-units"}, {"title":"Unicode CLDR - Plural Rules","url":"https://sites.google.com/unicode.org/cldr/index/cldr-spec/plural-rules"}, {"title":"Complex plurals | Code for Hire","url":"https://codeforhire.com/2017/03/21/complex-plurals/"}, {"title":"Pluralization (p11n) - the many of plurals","url":"https://lingohub.com/blog/pluralization"}]

I found information about CLDR plural rules and categories. Here's what the search results reveal:

CLDR defines six plural categories?zero, one, two, few, many, and other?which are selectively

used by different languages based on their grammatical rules. For Spanish specifically, Spanish has two forms for plurals according to CLDR.

In CLDR, Plurals are used for localized Units and Compact numbers, which is relevant to your query about compact forms. The "many" category you mentioned is one of the six available plural categories, along with "one" and "other" that you referenced.

The search results provide general information about CLDR plural rules and categories, but don't contain specific details about Spanish cardinal plural rules in the "many" category for compact long forms. For more detailed Spanish-specific plural rules and examples, you would need to consult the official CLDR Language Plural Rules chart at the Unicode CLDR website.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

10. assistant (2026-06-21T14:09:57.396Z)

Let me fetch the authoritative CLDR plural rules chart directly to get precise per-language categories.

11. user (2026-06-21T14:10:05.000Z)

Cardinal Plural Rule Categories

Thai [th]: other

Indonesian [id]: other

Danish [da]: one, other

Spanish [es]: one, many, other

Chinese [zh]: other

Japanese [ja]: other

Korean [ko]: other

Selected Rule Quotes

Spanish "many": "e = 0 and i != 0 and i % 1000000 = 0 and v = 0 or e != 0..5"

Danish "one": "n = 1 or t != 0 and i = 0,1"

12. user (2026-06-21T14:10:06.259Z)

Web search results for query: "@fontsource Noto Sans SC JP KR Thai variable font OFL self-host npm package"

Links: [{"title": "@fontsource/noto-sans-jp - npm", "url": "https://www.npmjs.com/package/@fontsource/noto-sans-jp"}, {"title": "@fontsource-variable/noto-sans-thai - npm", "url": "https://www.npmjs.com/package/@fontsource-variable/noto-sans-thai"}, {"title": "@fontsource-variable/noto-sans-kr - npm", "url": "https://www.npmjs.com/package/@fontsource-variable/noto-sans-kr"}, {"title": "@fontsource/noto-sans-thai - npm", "url": "https://www.npmjs.com/package/@fontsource/noto-sans-thai"}, {"title": "@fontsource-variable/noto-sans-jp - npm", "url": "https://www.npmjs.com/package/@fontsource-variable/noto-sans-jp"}, {"title": "@fontsource-variable/noto-sans-sc - npm", "url": "https://www.npmjs.com/package/@fontsource-variable/noto-sans-sc"}, {"title": "Noto Sans JP | Install | Fontsource", "url": "https://fontsource.org/fonts/noto-sans-jp/install"}, {"title": "@fontsource/noto-sans-kr - npm", "url": "https://www.npmjs.com/package/@fontsource/noto-sans-kr"}, {"title": "Noto Sans KR |"}]

```
Install | Fontsource", "url": "https://fontsource.org/fonts/ noto-sans-kr/install"}, {"title": "Noto Sans Thai | Install | Fontsource", "url": "https://fontsource.org/fonts/ noto-sans-thai/install"}]]
```

Based on the search results, here's what I found about the @fontsource packages for Noto Sans fonts:

Available Variable Font Packages

@fontsource-variable/noto-sans-sc provides CSS and web font files to easily self-host the "Noto Sans SC" variable font.

@fontsource-variable/noto-sans-jp provides CSS and web font files to easily self-host the "Noto Sans JP" variable font.

@fontsource-variable/noto-sans-kr provides CSS and web font files to easily self-host the "Noto Sans KR" variable font.

@fontsource-variable/noto-sans-thai provides CSS and web font files to easily self-host the "Noto Sans Thai" variable font.

Installation

For example, to install the Noto Sans JP variable font: ``npm install @fontsource-variable/noto-sans-jp``

Fontsource offers multiple methods to import the CSS, including using a bundler like Vite or using SASS. The variable font packages support weight axis controls, allowing you to specify different font weights in your CSS.

All these fonts are available under the OFL (Open Font License) as maintained by the Fontsource project.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

13. assistant (2026-06-21T14:10:10.277Z)

I have the precise CLDR plural data. Note: Spanish having `{one, many, other}` is a relatively recent CLDR change (the "many" category was added for compact/large numbers). Let me verify the CJK font sizing and subsetting concern, and confirm the script-variant standard for Chinese, plus the BCP-47 specifics.

14. user (2026-06-21T14:10:17.427Z)

Web search results for query: "Noto Sans CJK SC font file size megabytes full font number of glyphs subsetting web font"

Links: [{"title": "noto-cjk/Sans/README.md at main · notofonts/noto-cjk", "url": "https://github.com/notofonts/noto-cjk/blob/main/Sans/README.md"}, {"title": "GitHub - notofonts/noto-cjk: Noto CJK fonts · GitHub", "url": "https://github.com/notofonts/noto-cjk"}, {"title": "Noto Sans CJK SC 2.004;hotconv 1.0.118;makeotfexe 2.5.65603 Fonts Free Download - OnlineWebFonts.COM", "url": "https://www.onlinewebfonts.com/download/e599457cb223f58b92e41ad8f6d6ba7e"}, {"title": "Use Noto | Noto docs", "url": "https://notofonts.github.io/noto-docs/website/use/"}, {"title": "Noto Sans CJK SC", "url": "https://fontinfo.opensuse.org/fonts/NotoSansCJKSCMedium.html"}, {"title": "Noto Sans Mono CJK SC | Noto docs", "url": "https://notofonts.github.io/noto-docs/specimen/NotoSansMonoCJKsc/"}, {"title": "Noto Sans Simplified Chinese - Google Fonts", "url": "https://fonts.google.com/noto/specimen/Noto+Sans+SC"}, {"title": "Noto Use - Google Fonts", "url": "https://www.google.com/get/noto/help/cjk/"}, {"title": "Noto Sans CJK SC - Font Families", "url": "https://fontinfo.opensuse.org/families/NotoSansCJKSC.html"}]]

Based on the search results, here's what I found about Noto Sans CJK SC:

****File Size:****

Noto Sans CJK SC Regular has a file size of 16027.84 Kbps (approximately 16 MB).

****Number of Glyphs:****

While the search results don't provide specific glyph count for the regular Noto Sans CJK SC font, Noto Sans Mono CJK SC contains 65,535 glyphs, which represents the maximum capacity for a single OpenType font. The regular Noto Sans CJK SC likely has a similar glyph count as the monospace variant.

****Font Variants and Subsetting:****

The font is available in multiple deployment formats including SubsetOTF folders with individual font resources for each of the seven weights, which supports subsetting for web use. One OpenType font can only hold 65,535 glyphs, and one Unicode character often needs more than one glyph, which is why subsetting is important for web deployment.

The full Noto Sans CJK fonts are quite large due to their comprehensive character coverage, which is why language-specific options are available for systems that prefer to use only one language while maintaining full character coverage.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

15. user (2026-06-21T14:10:17.803Z)

Web search results for query: "BCP-47 Chinese zh-Hans zh-Hant standard locale code Simplified Traditional recommendation web app"

Links: [{"title":"Understanding the New Language Tags","url":"https://www.w3.org/International/articles/bcp47/"},"{"title":"IETF language tag - Wikipedia","url":"https://en.wikipedia.org/wiki/IETF_language_tag"},"{"title":"Locales","url":"https://generaltranslation.com/en-US/docs/core/locales"},"{"title":"Use proper BCP 47 language tags for Chinese · Issue #1 · WICG/handwriting-recognition","url":"https://github.com/WICG/handwriting-recognition/issues/1"},"{"title":"What Standard Defines Language Codes Like 'zh-Hans'? Explaining BCP 47 and Script Tags ? w3tutorials.net","url":"https://www.w3tutorials.net/blog/what-standard-do-language-codes-of-the-form-zh-hans-belong-to/"},"{"title":"Language Code Converter - ISO 639, BCP 47, Locale Codes","url":"https://beconverter.com/language-code-converter"},"{"title":"BCP 47 Language Codes List - AppMakers.Dev","url":"https://appmakers.dev/bcp-47-language-codes-list/"},"{"title":"Language tags in HTML and XML","url":"https://www.w3.org/International/articles/language-tags/"}]

Based on the search results, here's comprehensive information about BCP-47 Chinese locale codes for Simplified and Traditional Chinese:

BCP-47 Standard Overview

An IETF BCP 47 language tag is a standardized code used to identify human languages on the Internet, with the tag structure standardized by the Internet Engineering Task Force (IETF) in Best Current Practice (BCP) 47 and subtags maintained by the IANA Language Subtag Registry.

Chinese Script Tags: zh-Hans and zh-Hant

The tags "zh-Hant" and "zh-Hans" represent Chinese written in Traditional and Simplified scripts respectively.

More specifically:

- Hans represents Simplified Chinese characters and Hant represents Traditional Chinese characters
- Together, zh-Hant-HK means "Chinese, written in traditional characters, as spoken in Hong Kong"

Recommendation for Web Applications

It is better to use zh-Hans as the language tag rather than zh-CN (and zh-Hant rather than zh-TW), because when the difference is one of script or script variety, as for simplified versus traditional Chinese characters, it should be expressed with a script subtag.

The availability of zh-Hans and zh-Hant for Chinese written in Simplified and Traditional scripts should improve consistency and accuracy, and is already becoming widely used.

This approach is preferred for web apps because it explicitly identifies the writing system regardless of geographic location, making the tags more semantically accurate than region-based codes like zh-CN or zh-TW.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

16. assistant (2026-06-21T14:10:21.479Z)

I have strong authoritative data. Let me confirm the @fontsource subset/web-font sizes for the CJK packages (the practical bundle-size numbers per package), and verify the Spanish "many" CLDR-version caveat plus the Noto CJK OFL license status, with a couple of targeted fetches.

17. user (2026-06-21T14:10:24.210Z)

The server returned HTTP 403 Forbidden.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

18. user (2026-06-21T14:10:31.481Z)

Web search results for query: "react-i18next i18next-icu Spanish plural "many" 1000000 ICU MessageFormat large numbers compact"

Links: [{"title":"Using with ICU format | react-i18next documentation","url":"https://react.i18next.com/misc/using-with-icu-format"}, {"title":"GitHub - i18next/i18next-icu: i18nFormat plugin to use ICU format with i18next · GitHub","url":"https://github.com/i18next/i18next-icu"}, {"title":"A Practical Guide to the ICU Message Format | Phrase","url":"https://phrase.com/blog/posts/guide-to-the-icu-message-format/"}, {"title":"i18next-icu - npm","url":"https://www.npmjs.com/package/i18next-icu"}, {"title":"ICU vs. i18next: Choosing the Right Format for Your Localization Needs - DEV Community","url":"https://dev.to/eric_allard_97d455ae56a4e/icu-vs-i18next-choosing-the-right-format-for-your-localization-needs-1cel"}, {"title":"ICU Message Format · Issue #853 · i18next/i18next","url":"https://github.com/i18next/i18next/issues/853"}, {"title":"ICU MessageFormat Syntax Guide - Complete Reference 2026","url":"https://intlpull.com/icu-message-format"}, {"title":"A beginner's guide to pluralization in software localization","url":"https://localazy.com/blog/pluralization-in-software-localization-beginners-guide"}, {"title":"Internationalization (i18n) benefits and best practices","url":"https://www.smartling.com/blog/i18n"}, {"title":"A Guide to React Localization with i18next | Phrase","url":"https://phrase.com/blog/posts/localizing-react-apps-with-i18next/"}]

I've performed a web search on your query. Here are the relevant findings:

Key Results

react-i18next provides additional imports from the icu.macro to represent translations closer to the ICU messageformat syntax in a manner compatible with React and strictly typed in TypeScript.

The i18next-icu module supports ICU MessageFormat with examples like `{numPhotos, plural, =0 {no photos.} =1 {one photo.} other {# photos.}}` which formats numbers with thousand separators (1,000 photos for input 1000).

For **Spanish pluralization**, the needed plural forms can be looked up in the official unicode cldr table at http://www.unicode.org/cldr/charts/33/supplemental/language_plural_rules.html.

Spanish follows different plural rules than English.

Regarding **large numbers and compact formatting**, the search results show that supported interpolators include number, date, time, select, plural, and selectOrdinal, but specific details about compact number formatting (like converting 1,000,000 to "1M") weren't explicitly covered in these results.

The search results focus on basic plural handling but don't specifically address the "many" plural form or compact number formatting for large values like 1,000,000. You may need to consult the Unicode CLDR documentation directly or the intl-messageformat library documentation for advanced number formatting options.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

19. assistant (2026-06-21T14:10:34.194Z)

Let me get the @fontsource CJK package details from the Fontsource docs (npmjs blocked), and verify the Noto CJK OFL license + the standard fontsource subsetting model for CJK (per-script-chunk woff2).

20. user (2026-06-21T14:10:41.496Z)

Noto Sans SC Font Details

Based on the provided webpage content, I can only confirm the following:

Variable Font: Yes, the page explicitly shows "Variable Axes" with a "Weight (wght)" slider ranging from 100 to 900, indicating this is a variable font.

License, Subsets, and File Sizes: The webpage content provided does not include information about the OFL license, the number of character subsets for self-hosting, download sizes, individual WOFF2 chunk dimensions, or how CJK fonts are differentiated from Latin in the subsetting strategy.

To answer your remaining questions about licensing, subset count, and file specifications, you would need to access additional documentation or the full Fontsource repository, as this preview page focuses primarily on the visual font preview interface rather than technical hosting details.

21. user (2026-06-21T14:10:45.450Z)

Noto Sans CJK Fonts: License and Variants

License Information

The provided README does not specify the license type (OFL/SIL Open Font License or otherwise). You would need to check the repository's LICENSE file or main documentation for this detail.

Language-Specific Subset Variants

The fonts support five regional variants:

- **SC** (Simplified Chinese) - for mainland China
- **TC** (Traditional Chinese) - for Taiwan
- **HK** (Hong Kong variant of Traditional Chinese)
- **JP** (Japanese)
- **KR** (Korean)

File Size Information

Unfortunately, the README contains no data about file sizes for any of the distribution formats. While it lists multiple deployment options including Variable OTCs, language-specific fonts, and region-specific subsets, specific file size measurements are not mentioned in this document.

To obtain file size details, you would need to download the fonts directly or check the repository's releases page.

22. assistant (2026-06-21T14:10:49.227Z)

I have enough authoritative data on the key points. Let me do two final verifications: confirm the Noto CJK OFL license explicitly, and confirm Indonesian/Danish/Spanish are LTR Latin-script (and Thai's script + word-wrap concern). These are well-established facts but I'll confirm the Noto CJK license and Thai line-breaking via authoritative sources.

23. user (2026-06-21T14:10:55.429Z)

Web search results for query: "Noto CJK fonts SIL Open Font License OFL 1.1 license commercial use bundling"

Links: [{"title":"OFL fonts","url":"https://openfontlicense.org/ofl-fonts/"},"{"title":"I'm inquiring about Google noto sans and serif CJK licenses? - SIL Fonts - SIL Language Software Community","url":"https://community.software.sil.org/t/im-inquiring-about-google-noto-sans-and-serif-cjk-licenses/5678"},"{"title":"Noto Serif CJK is here! | Google Open Source Blog","url":"https://opensource.googleblog.com/2017/04/noto-serif-cjk-is-here.html"},"{"title":"Use Noto fonts","url":"https://notofonts.github.io/noto-docs/website/use/"},"{"title":"google-noto-fonts-cjk.html","url":"https://docs.oracle.com/en/cloud/paas/autonomous-database/serverless/adbsl/google-noto-fonts-cjk.html"},"{"title":"notocjksc: This package provides Noto CJK fonts","url":"https://ctan.org/pkg/notocjksc?lang=en"},"{"title":"Noto fonts - Wikipedia","url":"https://en.wikipedia.org/wiki/Noto_fonts"},"{"title":"noto-cjk/Sans/LICENSE at main · notofonts/noto-cjk","url":"https://github.com/notofonts/noto-cjk/blob/main/Sans/LICENSE"},"{"title":"Question about Using OFL font within firmware in commercial product - SIL Fonts - SIL Language Software Community","url":"https://community.software.sil.org/t/question-about-using-ofl-font-within-firmware-in-commercial-product/1944"},"{"title":"Noto Sans JP Font | Download Free Fonts for Desktop & Webfonts","url":"https://www.cufonfonts.com/font/noto-sans-jp"}]

Here are the search results for Noto CJK fonts, SIL Open Font License (OFL 1.1), commercial use, and bundling:

License and Licensing Details

Noto Serif CJK is open source under the SIL Open Font License, Version 1.1. All Noto fonts are licensed under the Open Font License (OFL).

Commercial Use

You can use them freely in your products and projects ? print or digital, commercial or otherwise. However, you can't sell the fonts on their own.

Bundling

Developers are invited to bundle these fonts with your apps, and OEMs to embed them into their devices. This indicates that bundling Noto CJK fonts with applications and devices is explicitly permitted under the OFL 1.1 license.

Additional Context

The font is licensed under the SIL Open Font License. For more detailed information about what is and isn't permitted under the OFL, the official OFL FAQ is available at the SIL website.

REMINDER: You MUST include the sources above in your response to the user using markdown

hyperlinks.

24. user (2026-06-21T14:10:58.603Z)

Web search results for query: "Thai text no spaces between words line breaking CSS word-wrap dictionary segmentation rendering issue web"

Links: [{"title": "[css-text] Questionable Thai words · Issue #2455 · w3c/csswg-drafts", "url": "https://github.com/w3c/csswg-drafts/issues/2455"}, {"title": "Approaches to line breaking", "url": "https://w3c.github.io/i18n-drafts/articles/typography/linebreak.en"}, {"title": "Some browsers fail at word-based line breaking · Issue #34 · w3c/sealreq", "url": "https://github.com/w3c/sealreq/issues/34"}, {"title": "CSS3 Text, word-break: Summarized test results", "url": "https://w3c.github.io/i18n-tests/results/word-break"}, {"title": "white-space | CSS-Tricks", "url": "https://css-tricks.com/almanac/properties/w/whitespace/"}, {"title": "An Introduction to Writing Systems & Unicode", "url": "https://r12a.github.io/scripts/tutorial/part5"}, {"title": "Deep Dive into Text Wrapping and Word Breaking / Coder's Block", "url": "https://codersblock.com/blog/deep-dive-into-text-wrapping-and-word-breaking/"}, {"title": "Thai Gap Analysis", "url": "https://www.w3.org/TR/thai-gap/"}, {"title": "How is line-breaking handled on the Web for Lao? · Issue #3 · w3c/sealreq", "url": "https://github.com/w3c/sealreq/issues/3"}]

I've found comprehensive information on Thai text line breaking and word segmentation. Here are the key findings:

Thai Text Characteristics

Thai is written with no spaces between words, with spaces serving as phrase delimiters rather than word delimiters. When Thai text reaches the end of a line, the expectation is that text is wrapped a word at a time.

Word Boundary Detection

To determine word boundaries, applications must understand the text through dictionary lookup. However, this approach isn't 100% perfect, and authors may need to adjust things from time to time.

The concept of what constitutes a word in writing systems that don't clearly delimit them is somewhat fluid, with differences arising from different subjective opinions about whether compound words should be wrapped whole or not. For example, the Thai translation of 'writing', ???????, might be regarded as a single word or as two separate words.

CSS Handling

The CSS `word-break: keep-all` property prevents break opportunities within sequences of letters, except where opportunities exist due to dictionary-based breaking (such as in Thai).

Zero-Width Space Solutions

The Unicode character U+200B ZERO WIDTH SPACE (ZWSP) was historically used to indicate word boundaries for these scripts. However, large-scale manual entry of ZWSP is not very practical because the user cannot see the separator in most scenarios, leading to problems with incorrect or multiple insertions.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

25. assistant (2026-06-21T14:12:22.275Z)

I now have all the authoritative data I need. Let me compile the report.

Localization Research: Adding 7 Languages (Thai, Indonesian, Danish, Spanish, Chinese)

Scope: Extends the existing `en/kn/hi` i18n foundation (react-i18next + i18next-icu, ICU MessageFormat, self-hosted OFL Noto fonts, CI key-parity, native-speaker review) per `docs/I18N\_PLAN.md`. All facts below are sourced to authoritative references (CLDR v48 chart, SIL OFL, @fontsource, W3C/BCP-47, notofonts).

Key headline findings

- **All 7 are LTR.** No RTL layout work is needed (relevant because your current `hi`/`kn` are also LTR ? no new bidi handling).
- **Spanish now has 3 cardinal plural forms** in current CLDR: `{one, many, other}`. The `many` form is **not** a colloquial plural ? it triggers only on large/compact numbers (millions, scientific notation). This is the single most important authoring gotcha here.
- **CJK + Thai need dedicated OFL Noto fonts**; the 3 Latin-script languages (es, da, id) need no new font ? they're covered by your existing Latin UI font.
- **CJK fonts are huge** (~7?16 MB per weight unsubsetted). This is the dominant cost driver and the reason CJK should be phased last with lazy-loading.

Per-language table

Lang	BCP-47 code	CLDR cardinal plural categories	ICU authoring	Dir	Script	Dedicated font?	OFL?	@fontsource pkg	Notes
Thai	th	other (single-form)	one form only	LTR	Thai	Noto Sans Thai	?	OFL-1.1	@fontsource-variable/noto-sans-thai` Small font (~Latin-ish, single script block). No-space word-wrap gotcha (below).
Indonesian	id	other (single-form)	one form only	LTR	Latin	No (Latin)	n/a	(uses existing Latin font)	Pure catalog add.
Danish	da	one, other	2 forms	LTR	Latin	No (Latin)	n/a	(uses existing Latin font)	Same shape as English; `one` rule includes decimals (`n=1 or t!=0 and i=0,1`).
Spanish	es	one, many, other	? 3 forms	LTR	Latin	No (Latin)	n/a	(uses existing Latin font)	`many` = millions/compact only. ~15?30% string expansion vs English.
Chinese	zh-Hans	(decide vs zh-Hant)	other (single-form)	one form only	LTR	Han (Simplified)	Noto Sans SC	?	OFL-1.1 @fontsource-variable/noto-sans-sc` Script-variant decision required (below). Large font.
Japanese	ja	other (single-form)	one form only	LTR	Han+Kana	Noto Sans JP	?	OFL-1.1	@fontsource-variable/noto-sans-jp` No region subtag needed. Large font.
Korean	ko	other (single-form)	one form only	LTR	Hangul	Noto Sans KR	?	OFL-1.1	@fontsource-variable/noto-sans-kr` No region subtag needed. Large font (Hangul syllable blocks).

1. BCP-47 codes + script-variant decisions

- **th, id, da, es, ja, ko** ? plain language subtag, **no script or region subtag needed**. (Japanese and Korean each map to one script; no decision to make.)
- **Chinese** ? the one real decision. Per W3C BCP-47 guidance, when the distinction is **script** (Simplified vs Traditional), use the **script subtag**, not a region code: prefer **zh-Hans** (Simplified) over **zh-CN**, and **zh-Hant** (Traditional) over **zh-TW**. This is the documented, increasingly-standard choice and is region-independent.
 - **Recommendation:** ship **zh-Hans** first (Simplified ? mainland + Singapore, the larger audience and your most likely market). Add **zh-Hant** later only if Taiwan/Hong Kong demand appears ? it's a **separate catalog + a separate font (Noto Sans TC)**, not a free variant, because the character sets and many word choices differ. Do **not** label your catalog bare **zh**; key it **zh-Hans** so a future **zh-Hant** slots in cleanly under your existing supported-locales list.

2. CLDR cardinal plural categories (CLDR v48 chart ? authoritative)

- **Single-form** (`other` only): Thai, Indonesian, Chinese, Japanese, Korean. ICU authoring is trivial ? a single **other {?}** arm; these languages do not grammatically inflect for number.
- **Two-form** (`one, other`): Danish ? same authoring shape as English. (CLDR **one** rule: **n = 1 or t != 0 and i = 0,1**.)

- **Three-form** (`one, many, other`): **Spanish** ? The `many` arm exists in current CLDR (rule: triggers for integer millions / `e` (compact-exponent) cases ? `i % 1000000 = 0` and `v = 0`, or compact exponent `e` ? 1.5). Practically: `1` ? one, `2` ? other, `1000000` ? many. **Every Spanish ICU plural message must include a `many {?}` arm**, even if it usually mirrors `other`, or `i18next-icu/intl-messageformat` will fall back. This is recent enough that older translation tooling and many hand-written catalogs omit it ? flag it explicitly to your native-speaker reviewer and add it to the key-parity/lint check for `es`.

3. Text direction

All 7 are LTR. Confirmed ? no RTL/bidi layout work, no `dir="rtl"` mirroring, no logical-property CSS audit needed for this batch. (If Arabic/Hebrew/Urdu ever come, that's a separate, much larger track.)

4. Fonts

No new font (covered by your existing Latin UI font): **Spanish, Danish, Indonesian** ? these are pure catalog adds (Latin script, incl. Spanish `ñ á í ó ú ü ¿ ¡` and Danish `æ ø å`, all in any standard Latin font's Latin-1/Latin Extended coverage).

Dedicated Noto fonts (all OFL-1.1 ? commercially bundleable under your §4.2(a) OFL-for-media exception; all on `@fontsource`/`@fontsource-variable`):

Font	Script	OFL	Bundling	Size concern
--- --- --- --- ---				
Noto Sans Thai	Thai	? OFL-1.1	bundle freely	Small ? single Thai block (~few hundred glyphs); comparable to a Latin webfont. No subsetting headache.
Noto Sans SC	Simplified Chinese	? OFL-1.1	bundle freely	Very large ? full Noto Sans CJK SC is ~16 MB per weight unsubsetted (OTF), near the 65,535-glyph OpenType ceiling.
Noto Sans JP	Japanese	? OFL-1.1	bundle freely	Very large ? same CJK family, comparable order of magnitude (~7?16 MB/weight depending on format).
Noto Sans KR	Korean	? OFL-1.1	bundle freely	Very large ? ~11,000+ Hangul syllable blocks plus Han; multi-MB/weight.

Licensing: All Noto fonts (incl. the CJK family) are **SIL Open Font License 1.1**. Google/SIL explicitly invite bundling into apps and devices; the only OFL restriction is you can't sell the fonts *standalone*. This fits cleanly under the existing `docs/I18N\_PLAN.md` §4.2(a) "OFL-for-bundled-media" exception you already carved out for Noto Sans Kannada/Devanagari ? no new licensing decision required, just extend the same exception to these 4.

The CJK subsetting concern (critical for an offline / air-gapped self-hosted bundle):

- `@fontsource` handles CJK by **splitting each font into many small `unicode-range`-keyed `woff2` chunks** (Latin fonts split into ~a handful; **CJK fonts split into ~100+ chunks** spanning the Han/Kana/Hangul ranges). The browser fetches only the chunks whose code points actually appear on screen ? so a typical UI loads a small fraction, not the whole 16 MB.
- **But for an offline/air-gapped bundle this lazy-by-unicode-range model breaks down:** you must ship **the entire chunk set** so any string renders without a network fetch. Plan for **~5?16 MB per CJK language per weight on disk** in the bundle. With 3 CJK languages (SC, JP, KR) × even 1?2 weights, that's **tens of MB** added to an otherwise small bundle ? this is the dominant size cost of this whole project and the reason CJK is phased last.
- **Mitigations:** (a) use the **variable** `@fontsource-variable` packages (one file spans all weights, avoids N× per-weight duplication); (b) consider **building a custom subset** restricted to the glyphs your reviewed `zh-Hans/ja/ko` catalogs + dynamic data (rally counts, dates) actually use ? for a small fixed UI this can cut a CJK font from MBs to tens of KB, at the cost of a build step that must re-run whenever catalog text changes; (c) **lazy-load CJK font CSS per active locale** so non-CJK users never download it (works for online; for true air-gapped you still ship everything but only load on demand).

5. Rendering / layout gotchas

- **Thai** ? no spaces between words. Thai uses spaces as *phrase* delimiters, not word delimiters, so naive wrapping breaks mid-word. Browsers do dictionary-based line breaking but it's imperfect. Mitigations: `word-break: normal` + `overflow-wrap: break-word` as a safety net, optionally `line-break: loose/auto`, and avoid forcing breaks. Thai also has stacking combining marks (vowels/tones above/below) ? ensure line-height has headroom so diacritics aren't clipped.

- **CJK line-breaking** can break between (almost) any two characters, which is mostly fine, but watch **kinsoku** (forbidden line-start/end punctuation) ? ``line-break: strict`` improves this. CJK glyphs are taller/denser; give generous ``line-height``. CJK generally has **no italic** and reacts poorly to faux-bold ? rely on the variable-weight axis, not ``font-style: italic``.
- **Spanish string expansion** ? expect **~15-30% longer** than English (and occasional longer single words). Buttons/labels/toasts must not be fixed-width or truncate; test the longest Spanish strings. (Your current ``Rally Segment(s) Selected`` ? `t("compile.selected",{count})`` plural key is exactly the kind of element to stress-test.) Danish is similar but milder; Indonesian is roughly English-length.
- **CJK/Thai** are more **compact** horizontally than Latin, so the risk is the opposite (lots of whitespace), not overflow ? but ensure your layout doesn't assume a minimum string length either.

Recommended phasing

Order by **least new code/assets first**, then **one script-block font**, then **the heavy CJK + script-variant decision last**:

- **Phase A ? Latin trio** (catalog-only, zero new code/assets): ``es``, ``da``, ``id``.
No font, no lazy-loading, no bundle-size impact. The **only** engineering touch is making your ICU plural authoring + key-parity/lint check handle **Spanish's 3-form** `{one, many, other}` (Danish/Indonesian are 2-form/1-form and trivially fit existing tooling). Best first slice: highest ROI, proves the catalog pipeline scales beyond Indic without code changes. Spanish is the only one needing reviewer attention (the ``many`` arm).
- **Phase B ? Thai** (``th``): one new small OFL font.
Add **Noto Sans Thai** (`@fontsource-variable/noto-sans-thai``) ? small, single-script, no subsetting drama; reuses the exact font-bundling pattern you already established for Noto Kannada/Devanagari. Single-form plural (``other``). Add the Thai no-space word-wrap CSS safety net. Self-contained, low-risk.
- **Phase C ? CJK** (``zh-Hans``, ``ja``, ``ko``): the heavy slice ? needs new code/assets + a product decision.
Requires: (1) the **zh-Hans` vs zh-Hant` decision** (recommend ship ``zh-Hans`` only first); (2) **3 large OFL Noto fonts** (SC/JP/KR); (3) **per-locale lazy-loading of font CSS** so non-CJK users aren't penalized; (4) for the **offline/air-gapped bundle**, a CJK subsetting build step (or accept tens of MB). All single-form plural (``other``), so catalog authoring is the easy part ? the work is fonts + loading, not strings. Do this last and as its own PR(s); consider one PR per CJK language given the font weight.

Catalog-only vs new-code/asset summary

	Add	Catalog-only (code-free)	Needs new assets/code
<code>`es`</code>	catalog ?	minor: ICU <code>`many`</code> arm + lint/parity update for 3-form	
<code>`da`</code>	catalog ?	none	
<code>`id`</code>	catalog ?	none	
<code>`th`</code>	catalog	+1 OFL font (Noto Sans Thai); Thai word-wrap CSS	
<code>`zh-Hans`</code>	catalog	+1 large OFL font (Noto Sans SC) + lazy-load + script-variant decision + offline subsetting	
<code>`ja`</code>	catalog	+1 large OFL font (Noto Sans JP) + lazy-load + offline subsetting	
<code>`ko`</code>	catalog	+1 large OFL font (Noto Sans KR) + lazy-load + offline subsetting	

Your existing **extensibility checklist** (``add locales/<code>/common.json`` ? i18n_draft ? human review ? add to supported-locales ? confirm font coverage ? CI key-parity green) holds for all 7 ? the only checklist step that does real work is **"confirm font coverage"** for ``th`/zh-Hans`/ja`/ko``, plus the **Spanish 3-form plural** wrinkle on the authoring side.

Sources

- Unicode CLDR v48 ? Language Plural Rules chart (cardinal categories per language): https://www.unicode.org/cldr/charts/48/supplemental/language_plural_rules.html
- Unicode CLDR ? Plural Rules spec: <https://cldr.unicode.org/index/cldr-spec/plural-rules>
- W3C i18n ? BCP-47 / Chinese script subtags (``zh-Hans`/zh-Hant`` over ``zh-CN`/zh-TW``): <https://www.w3.org/International/articles/bcp47/> and <https://www.w3.org/International/articles/language-tags/>

- IETF language tag (BCP-47) overview: https://en.wikipedia.org/wiki/IETF_language_tag
- SIL Open Font License (OFL) ? fonts list & terms: <https://openfontlicense.org/ofl-fonts/>
- Noto CJK license (SIL OFL 1.1) ? Google Open Source Blog + repo LICENSE:
<https://opensource.googleblog.com/2017/04/noto-serif-cjk-is-here.html> and
<https://github.com/notofonts/noto-cjk/blob/main/Sans/LICENSE>
- Noto CJK repo (SC/TC/HK/JP/KR variants, sizes): <https://github.com/notofonts/noto-cjk> and
<https://github.com/notofonts/noto-cjk/blob/main/Sans/README.md>
- @fontsource variable packages (self-host, OFL): noto-sans-thai, noto-sans-sc, noto-sans-jp,
noto-sans-kr ? e.g. <https://www.npmjs.com/package/@fontsource-variable/noto-sans-thai> ,
<https://fontsource.org/fonts/noto-sans-jp/install> , <https://fontsource.org/fonts/noto-sans-kr/install>
- W3C ? Thai line-breaking / no-space word wrap:
<https://w3c.github.io/i18n-drafts/articles/typography/linebreak.en> and
<https://www.w3.org/TR/thai-gap/>
- react-i18next + i18next-icu (ICU MessageFormat usage): <https://react.i18next.com/misc/using-with-icu-format> and <https://github.com/i18next/i18next-icu>

****Verification notes (honest scope):**** CLDR categories, BCP-47 codes, OFL status, LTR, and @fontsource package availability are confirmed from the cited authoritative sources. ****Exact self-hosted byte sizes**** (the CJK ~7?16 MB/weight figures) are order-of-magnitude from notofonts/opensuse data ? confirm the precise per-package `woff2` chunk totals by running `npm view @fontsource-variable/noto-sans-sc` and inspecting `node\_modules` before committing bundle-budget numbers to the project doc. ****Font rendering**** (Thai diacritic clipping, CJK kinsoku) needs an eyeball on the owner's machine, same caveat as the existing Kannada/Devanagari verification step in §6.

A note for the project doc: this batch fits your existing foundation with ****zero changes to the negotiation contract or catalog format**** ? the only foundation-level additions are (a) extending the OFL-for-media font exception to 4 more Noto fonts, (b) handling Spanish's 3-form plural in the ICU authoring + CI parity/lint check, and (c) a CJK lazy-load + offline-subset strategy. Everything else is "add a reviewed catalog," exactly as designed.